

We now have all the necessary Docker images to move further to the next sections.

Quick programming shell

Our first shell is pshell, which provides an interactive environment with the filesystem, vim and C compiler. This shell is provided as a firmware to flash the Pico boards. Download its latest tarball from the releases link provided in the 'Reference' section at the end of this article. Now untar the downloaded .tgz and you should be able to see various .uf2 firmware files in the directory to flash your Pico board. The firmware files ending with _usb.uf2 will be connected to the Pico board using the minicom serial terminal.

To kick things off, flash your Pico board by connecting its USB cable to your computer. Press the **BOOTSET** button while inserting the cable into the microUSB port of the Pico board. You should see the Pico board mounted as a USB device on your computer. Just copy the appropriate *usb.uf2* firmware downloaded to the mounted Pico drive. You'll see the *uf2* file disappearing from the mounted Pico drive once programming of the new firmware is complete. Now your Pico device will automatically reset and be ready for use. Next, connect your Pico board over a serial link by executing the command `docker run -it --rm --device=/dev/ttyACM0:/dev/ttyACM0 minicomscreen minicom -D /dev/ttyACM0` on your machine. Please note, the port may not be `/dev/ttyACM0`, depending upon the other Pico boards connected to your

machine. So adjust for that number in the command issued. The command should present a screen similar to the one shown in Figure 1. Just press *Enter* and you should see all the commands provided by the pshell.

The pshell provides an autocomplete feature, and you can execute a clear command anytime to keep your shell looking tidy. Execute the *status* command to see the Pico storage, memory and console statistics. For *IX command line users, the *cat*, *cd*, *cp*, *ls*, *mkdir*, *mv*, *rm*, etc, commands should be familiar. The *reboot* command is helpful whenever there is a need to start your shell from a clean slate.

It's time to see the programming powers of pshell through the official examples provided. Execute the command `vi blink.c` and save after entering the C code given below:

```
/* --- Start of blink.c --- */

int main() {
    int led_pin = PICO_DEFAULT_LED_PIN;
    gpio_init(led_pin);
    gpio_set_dir(led_pin, GPIO_OUT);
    int tic = 0;
    for (;;) {
        gpio_put(led_pin, tic ^= 1);
        if (getchar_timeout_us(500000) == 3)
            break;
    }
    gpio_put(led_pin, 0);
    return 0;
}

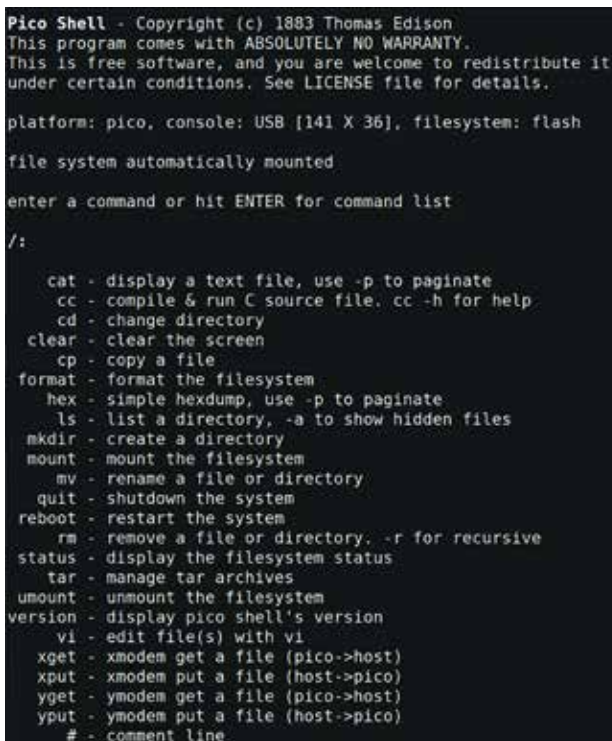
/* --- End of blink.c --- */
```

Now, execute the command `cc blink.c` to blink the LED on your Pico board. Can the C programming for a Pico board be simpler than this? You can interrupt the LED blink execution by pressing the *Ctrl* and *C* keys. Executing the command `cc -h` will dump all the options and libraries supported by it. Quickly try to execute the *umount* command, and the *ls* command will show no files. Executing the *mount* command followed by the *ls* command will show your created files again.

Let's move ahead and have some fun with your Pico board PWM (pulse width modulation) functionality. Create *fade.c* and execute it to see the onboard LED fading and lighting up alternately.

```
int fade, slice, going_up;

void on_pwm_wrap() {
    pwm_clear_irq(slice);
    if (going_up) {
```



```
Pico Shell - Copyright (c) 1883 Thomas Edison
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions. See LICENSE file for details.

platform: pico, console: USB [141 X 36], filesystem: flash
file system automatically mounted

enter a command or hit ENTER for command list
/:
  cat - display a text file, use -p to paginate
  cc - compile & run C source file. cc -h for help
  cd - change directory
  clear - clear the screen
  cp - copy a file
  format - format the filesystem
  hex - simple hexdump, use -p to paginate
  ls - list a directory, -a to show hidden files
  mkdir - create a directory
  mount - mount the filesystem
  mv - rename a file or directory
  quit - shutdown the system
  reboot - restart the system
  rm - remove a file or directory. -r for recursive
  status - display the filesystem status
  tar - manage tar archives
  umount - unmount the filesystem
  version - display pico shell's version
  vi - edit file(s) with vi
  xget - xmodem get a file (pico->host)
  xput - xmodem put a file (host->pico)
  yget - ymodem get a file (pico->host)
  yput - ymodem put a file (host->pico)
  # - comment line
```

Figure 1: pshell terminal